COVER FEATURE

Can We Make Operating Systems Reliable and Secure?

Andrew S. Tanenbaum, Jorrit N. Herder, and Herbert Bos
Vrije Universiteit, Amsterdam

Microkernels—long discarded as unacceptable because of their lower performance compared with monolithic kernels—might be making a comeback in operating systems due to their potentially higher reliability, which many researchers now regard as more important than performance.

When was the last time your TV set crashed or implored you to download some emergency software update from the Web? After all, unless it is an ancient set, it is just a computer with a CPU, a big monitor, some analog electronics for decoding radio signals, a couple of peculiar I/O devices—a remote control, a built-in VCR or DVD drive—and a boatload of software in ROM.

This rhetorical question points out a nasty little secret that we in the computer industry do not like to discuss: Why are TV sets, DVD recorders, MP3 players, cell phones, and other software-laden electronic devices reliable and secure but computers are not? Of course there are many "reasons"—computers are flexible, users can change the software, the IT industry is immature, and so on—but as we move to an era in which the vast majority of computer users are nontechnical people, increasingly these seem like lame excuses to them.

What consumers expect from a computer is what they expect from a TV set: You buy it, you plug it in, and it works perfectly for the next 10 years. As IT professionals, we need to take up this challenge and make computers as reliable and secure as TV sets.

The worst offender when it comes to reliability and security is the operating system. Although application programs contain many flaws, if the operating system were bug free, bugs in application programs could do only limited damage, so we will focus here on operating systems.

However, before getting into the details, a few words about the relationship between reliability and security are in order. Problems with each of these domains often have the same root cause: bugs in the software. A buffer overrun error can cause a system crash (reliability problem), but it can also allow a cleverly written virus or worm to take over the computer (security problem). Although we focus primarily on reliability, improving reliability can also improve security.

WHY ARE SYSTEMS UNRELIABLE?

Current operating systems have two characteristics that make them unreliable and insecure: They are huge and they have very poor fault isolation. The Linux kernel has more than 2.5 million lines of code; the Windows XP kernel is more than twice as large.

One study of software reliability showed that code contains between six and 16 bugs per 1,000 lines of executable code,[1] while another study put the fault density at two to 75 bugs per 1,000 lines of executable code,[2] depending on module size. Using a conservative estimate of six bugs per 1,000 lines of code, the Linux kernel probably has something like 15,000 bugs; Windows XP has at least double that.

To make matters worse, typically, about 70 percent of the operating system consists of device drivers, which have error rates three to seven times higher than ordinary code,[3] so the bug counts cited above are probably gross underestimates. Clearly, finding and correcting all

Published by the IEEE Computer Society
0018-9162/06/$20.00 © 2006 IEEE

these bugs is simply not feasible; furthermore, bug fixes frequently introduce new bugs.

The large size of current operating systems means that no one person can understand the whole thing. Clearly, it is difficult to engineer a system well when nobody really understands it.

This brings us to the second issue: fault isolation. No single person understands everything about how an aircraft carrier works either, but the subsystems on an aircraft carrier are well isolated. A problem with a clogged toilet cannot affect the missile-launching subsystem.

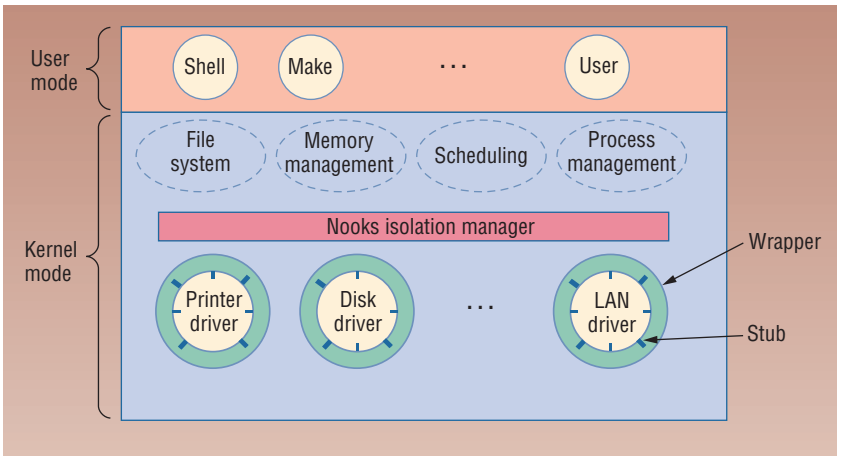

Figure 1. The Nooks model. Each driver is wrapped in a layer of protective software that monitors all interactions between the driver and the kernel.

Operating systems do not have this kind of isolation between components. A modern operating system contains hundreds or thousands of procedures linked together as a single binary program running in kernel mode. Every single one of the millions of lines of kernel code can overwrite key data structures that an unrelated component uses, crashing the system in ways difficult to detect. In addition, if a virus or worm infects one kernel procedure, there is no way to keep it from rapidly spreading to others and taking control of the entire machine.

Going back to our ship analogy, modern ships have multiple compartments within the hull; if one compartment springs a leak, only that one is flooded, not the entire hull. Current operating systems are like ships before compartmentalization was invented: Every leak can sink the ship.

Fortunately, the situation is not hopeless. Researchers are endeavoring to produce more reliable operating systems. Here we address four different approaches that researchers are using to make future operating systems more reliable and secure, proceeding from the least radical to the most radical solution.

ARMORED OPERATING SYSTEMS

The most conservative approach, Nooks,[4] is designed to improve the reliability of existing operating systems such as Windows and Linux. Nooks maintains the monolithic kernel structure, with hundreds or thousands of procedures linked together in a single address space in kernel mode, but it focuses on making device drivers—the core of the problem—less dangerous.

In particular, as Figure 1 shows, Nooks protects the kernel from buggy device drivers by wrapping each driver in a layer of protective software to form a lightweight protection domain, a technique sometimes called sandboxing. The wrapper around each driver carefully monitors all interactions between the driver and the kernel. This technique can also be used for other extensions to the kernel such as loadable file systems, but for simplicity we will just refer to drivers.

The Nooks project's goals are to

- protect the kernel against driver failures,
- recover automatically when a driver fails, and
- do all of this with as few changes as possible to existing drivers and the kernel.

Protecting the kernel against malicious drivers is not a goal. The initial implementation was on Linux, but the ideas apply equally well to other legacy kernels.

Isolation

The main tool used to keep faulty drivers from trashing kernel data structures is the virtual memory page map. When a driver runs, all pages outside it are changed to read-only, thus implementing a separate lightweight protection domain for each driver. In this way, the driver can read the kernel data structures it needs, but any attempt to directly modify a kernel data structure results in a CPU exception that the Nooks isolation manager catches. Access to the driver's private memory, where it stores stacks, a heap, private data structures, and copies of kernel objects, is read-write.

Interposition

Each driver class exports a set of functions that the kernel can call. For example, sound drivers might offer a call to write a block of audio samples to the card, another one to adjust the volume, and so on. When the driver is loaded, an array of pointers to the driver's functions is filled in, so the kernel can find each one. In addition, the driver imports a set of functions provided by the kernel, for example, for allocating a data buffer.

Nooks provides wrappers for both the exported and imported functions. When the kernel now calls a driver

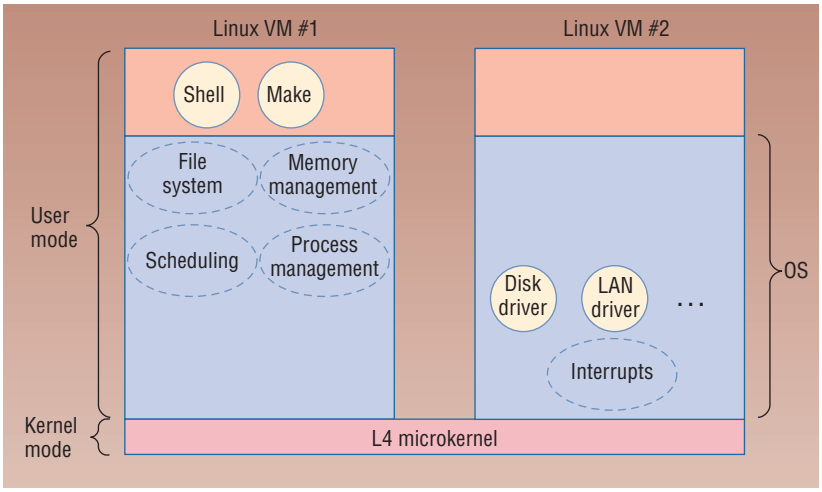

Figure 2. Virtual machines. One of the virtual Linux machines runs the application programs while one or more other machines run the device drivers.

function or a driver calls a kernel function, the call actually goes to a wrapper that checks the parameters for validity and manages the call. While the wrapper stubs—shown in Figure 1 as lines sticking into and out of the drivers—are generated automatically from their function prototypes, developers must handwrite the wrapper bodies. In all, the Nooks team wrote 455 wrappers: 329 for functions the kernel exports and 126 for functions the device drivers export.

When a driver tries to modify a kernel object, its wrapper copies the object into the driver's protection domain, that is, onto its private read-write pages. The driver then modifies the copy. Upon successful completion of the request, the isolation manager copies modified kernel objects back to the kernel. In this way, a driver crash or failure during a call always leaves kernel objects in a valid state. Keeping track of imported objects is object specific, so the Nooks team had to handwrite code to track the 43 classes of objects the Linux drivers use.

Recovery

After a failure, the user-mode recovery agent runs and consults a configuration database to see what to do. In many cases, releasing any resources held and restarting the driver is enough because most common algorithmic bugs are usually found in testing, leaving mostly timing and uncommon bugs.

This technique can recover the system, but running applications can fail. In additional work,[5] the Nooks team added the concept of shadow drivers to allow applications to continue after a driver failure.

In short, during normal operation, a shadow driver logs communication between each driver and the kernel if it will be needed for recovery. After a driver restart, the shadow driver feeds the newly restarted driver from the log—for example, repeating the I/O control (IOCTL) system call to set parameters such as audio volume. The kernel is unaware of the process of getting the new driver back into the same state the old one was in. Once this is accomplished, the driver begins processing new requests.

Limitations

While experiments show that Nooks can catch 99 percent of the fatal driver errors and 55 percent of the nonfatal ones, it is not perfect. For example, drivers can execute privileged instructions they should not execute; they can write to incorrect I/O ports; and they can get into infinite loops. Furthermore, the Nooks team had to write large numbers of wrappers manually, and they could contain faults. Finally, drivers are not prevented from reenabling write access to all of memory. Nevertheless, it is potentially a useful step toward improving the reliability of legacy kernels.

PARAVIRTUAL MACHINES

A second approach has its roots in the virtual machine concept, which goes back to the late 1960s.[6] In short, the idea is to run a special control program, called a virtual machine monitor, on the bare hardware instead of an operating system. The virtual machine creates multiple instances of the true machine. Each instance can run any software the bare machine can.

This technique is commonly used to allow two or more operating systems, say Linux and Windows, to run on the same hardware at the same time, with each one thinking it has the entire machine to itself. The use of virtual machines has a well-deserved reputation for good fault isolation—after all, if none of the virtual machines even know about the other ones, problems in one machine cannot spread to others.

The research here is to adapt this concept to protection within a single operating system, rather than between different operating systems.[7] Furthermore, because the Pentium is not fully virtualizable, a concession was made to the idea of running an unmodified operating system in the virtual machine. This concession allows modifications to be made to the operating system to make sure it does not do anything that cannot be virtualized. To distinguish it from true virtualization, this technique is called *paravirtualization*.

Specifically, in the 1990s, a research group at the University of Karlsruhe built the L4 microkernel.[8] They were able to run a slightly modified version of Linux (L^4Linux) on top of L4 in what could be described as a kind of virtual machine.[9] The researchers later realized that instead of running only one copy of Linux on L4, they could run multiple copies. As Figure 2 shows, this

insight led to the idea of having one of the virtual Linux machines run the application programs while one or more other machines run the device drivers.

By putting the device drivers in one or more virtual machines separated from the main virtual machine running the rest of the operating system and the application programs, if a device driver crashes, only its virtual machine goes down, not the main one. An additional advantage of this approach is that the device drivers do not have to be modified as they see a normal Linux kernel environment. Of course, the Linux kernel itself had to be modified to achieve paravirtualization, but this is a one-time change, and it is not necessary to repeat it for each device driver.

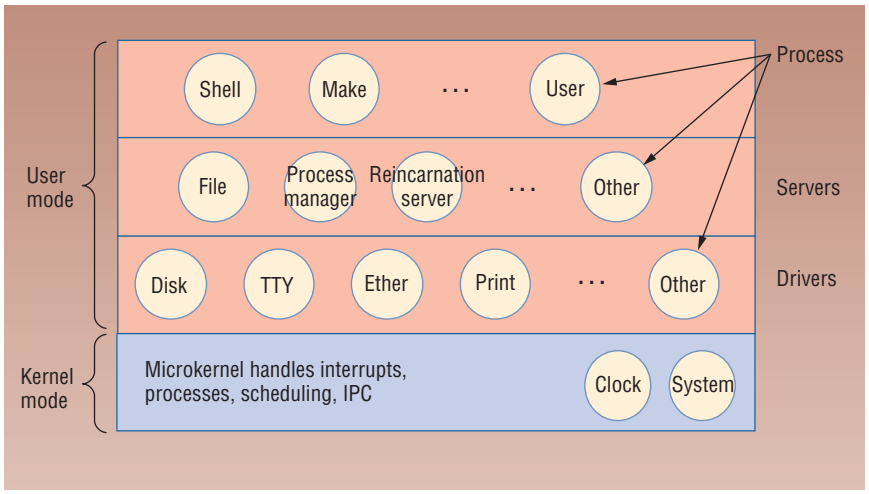

Figure 3. The Minix 3 architecture. The microkernel handles interrupts, provides the basic mechanisms for process management, implements interprocess communication, and performs process scheduling.

Since the device drivers are running in the hardware's user mode, a major issue is how they actually perform I/O and handle interrupts. Physical I/O is handled by adding about 3,000 lines of code to the Linux kernel on which the drivers run to allow them to use the L4 services for I/O instead of doing it themselves. An additional 5,000 lines of code handle communication between the three isolated drivers—disk, network, and PCI bus—and the virtual machine running the application programs.

In principle, this approach should provide greater reliability than a single operating system because when a virtual machine containing one or more drivers crashes, the virtual machine can be rebooted and the drivers returned to their initial state. No attempt is made to return drivers to their previous (precrash state) as in Nooks. Thus, if an audio driver crashes, it will be restored with the sound level set to the default, rather than to the level it had prior to the crash.

Performance measurements have shown that the overhead of using paravirtualized machines in this fashion is about 3 to 8 percent.

MULTISERVER OPERATING SYSTEMS

The first two approaches focus on patching legacy operating systems. The next two focus on future systems.

One of these approaches directly attacks the core of the problem: having the entire operating system run as a single gigantic binary program in kernel mode. Instead, only a tiny microkernel runs in kernel mode with the rest of the operating system running as a collection of fully isolated user-mode server and driver processes.

This idea has been around for 20 years, but it was not fully explored the first time around because it has slightly lower performance than a monolithic kernel. In the 1980s, performance counted for everything, and reliability and security were not yet on the radar. Of course, at the time, aeronautical engineers did not worry too much about miles per gallon or the ability of cockpit doors to withstand armed attacks. Times change, and people's ideas of what is important change too.

Multiserver architecture

Taking a look at a modern example helps to make the idea of a multiserver operating system clearer. As Figure 3 shows, in Minix 3, the microkernel handles interrupts, provides the basic mechanisms for process management, implements interprocess communication, and performs process scheduling. It also offers a small set of kernel calls to authorized drivers and servers, such as reading a selected portion of a specific user's address space or writing to authorized I/O ports. The clock driver shares the microkernel's address space, but it is scheduled as a separate process. No other drivers run in kernel mode.

Above the microkernel is the device driver layer.[10] Each I/O device has its own driver that runs as a separate process in its own private address space, protected by the memory management unit (MMU) hardware. The layer includes driver processes for the disk, terminal (keyboard and display), Ethernet, printer, audio, and so on. The drivers run in user mode and cannot execute privileged instructions or read or write the computer's I/O ports; they must make kernel calls to obtain these services. While introducing a small amount of overhead, this design also enhances reliability.

On top of the device driver layer is the server layer. The file server is a small (4,500 lines of executable code) program that accepts requests from user processes for

the Posix system calls relating to files, such as read, write, lseek, and stat and carries them out. Also in this layer is the process manager, which handles process and memory management and carries out Posix and other system calls such as fork, exec, and brk.

A somewhat unusual feature is the reincarnation server, which is the parent process of all the other servers and all the drivers. If a driver or server crashes, exits, or fails to respond to the periodic pings, the reincarnation server kills it if necessary and then restarts it from a copy on disk or in RAM. Drivers can be restarted this way, but currently only servers that do not maintain much internal state can be restarted.

Other servers include the network server, which contains a complete TCP/IP stack; the data store, a simple name server that the other servers use; and the information server, which aids debugging.

Finally, located above the server layer are the user processes. The only difference between this and other Unix systems is that the library procedures for read, write, and the other system calls do their work by sending messages to servers. Other than this difference—hidden in the system libraries—they are normal user processes that can use the Posix API.

Interprocess communication

Because it allows all processes to cooperate, interprocess communication (IPC) is of crucial importance in a multiserver operating system. However, since all servers and drivers in Minix 3 run as physically isolated processes, they cannot directly call each other's functions or share data structures. Instead, Minix 3 performs IPC by passing fixed-length messages using the rendezvous principle: When both the sender and the receiver are ready, the system copies the message directly from the sender to the receiver. In addition, an asynchronous event notification mechanism is available. Events that cannot be delivered are marked pending a bitmap in the process table.

Minix 3 elegantly integrates interrupts with the message passing system. Interrupt handlers use the notification mechanism to signal I/O completion. This mechanism allows a handler to set a bit in the driver's "pending interrupts" bitmap and then continue without blocking. When the driver is ready to receive the interrupt, the kernel turns it into a normal message.

Reliability features

Minix 3's reliability comes from multiple sources. First, only about 4,000 lines of code run in the kernel, so with a conservative estimate of six bugs per 1,000 lines, the total number of bugs in the kernel is probably only about 24—compared with 15,000 for Linux and far more for Windows. Since all device drivers except the clock are user processes, no foreign code ever runs in kernel mode. The kernel's small size also could make it practical to verify its code, either manually or by formal techniques.

Minix 3's IPC design does not require message queuing or buffering, which eliminates the need for buffer management in the kernel. Furthermore, since IPC is a powerful construct, the IPC capabilities of each server and driver are tightly confined. For each process, the available IPC primitives, allowed destinations, and user event notifications are restricted. User processes, for example, can use only the rendezvous principle and can send to only the Posix servers.

Minix 3 elegantly integrates interrupts with the message passing system.

In addition, all kernel data structures are static. All of these features greatly simplify the code and eliminate kernel bugs associated with buffer overruns, memory leaks, untimely interrupts, untrusted kernel code, and more. Of course, moving most of the operating system to user mode does not eliminate the inevitable bugs in drivers and servers, but it renders them far less powerful. A kernel bug can trash critical data structures, write garbage to the disk, and so on; a bug in most drivers and servers cannot do as much damage since these processes are strongly compartmentalized, and they are very restricted in what they can do.

The user-mode drivers and servers do not run as superuser. They cannot access memory outside their own address spaces except by making kernel calls (which the kernel inspects for validity). Stronger yet, bitmaps and ranges within the kernel's process table control the set of permitted kernel calls, IPC capabilities, and allowed I/O ports on a per-process basis. For example, the kernel can prevent the printer driver from writing to user address spaces, touching the disk's I/O ports, or sending messages to the audio driver. In traditional monolithic systems, any driver can do anything.

Another reliability feature is the use of separate instruction and data spaces. Should a bug or virus manage to overrun a driver or server buffer and place foreign code in data space, the injected code cannot be executed by jumping to it or having a procedure return to it, since the kernel will not run code unless it is in the process's (read-only) instruction space.

Among the other specific features aimed at improving reliability, the most crucial is the self-healing property. If a driver does a store through an invalid pointer, gets into an infinite loop, or otherwise misbehaves, the reincarnation server will automatically replace it, often without affecting running processes.

While restarting a logically incorrect driver will not remove the bug, in practice subtle timing and similar

bugs cause many problems, and restarting the driver will often repair the system. In addition, this mechanism allows recovery from failures that are caused by attacks, such as the "ping of death," which can crash a computer by sending it an incorrectly formatted IP packet.

Performance considerations

For decades, researchers have criticized multiserver architectures based on microkernels because of alleged performance problems. However, various projects have proven that modular designs actually can provide competitive performance. Despite the fact that Minix 3 has not been optimized for performance, the system is reasonably fast. The performance loss that user-mode drivers cause compared to in-kernel drivers is less than 10 percent, and the system can build itself, including the kernel, common drivers, and all servers (112 compilations and 11 links) in less than 6 seconds on a 2.2-GHz Athlon processor.

The fact that multiserver architectures make it possible to provide a highly reliable Unix-like environment at the cost of only a small performance overhead makes this approach practical. Minix 3 for the Pentium is available for free download under the Berkeley license at www.minix3.org. Ports to other architectures and to embedded systems are under development.

LANGUAGE-BASED PROTECTION

The most radical approach comes from an unexpected source—Microsoft Research. In effect, the Microsoft approach discards the concept of an operating system as a single program running in kernel mode plus some collection of user processes running in user mode, and replaces it with a system written in new type-safe languages that do not have all the pointer and other problems associated with C and C++. Like the previous two approaches, this one has been around for decades.

The Burroughs B5000 computer used this approach. The only language available then was Algol, and protection was handled not by an MMU—which the machine did not have—but by the Algol compiler's refusal to generate "dangerous" code. Microsoft Research's approach updates this idea for the 21st century.

Overview

This system, called Singularity, is written almost entirely in Sing#, a new type-safe language. This language is based on C#, but augmented with message passing primitives whose semantics are defined by formal, written contracts. Because language safety tightly constrains the system and user processes, all processes can run together in a single virtual address space. This design leads to both safety—because the compiler will not allow a process to touch another process's data—and efficiency—because it eliminates kernel traps and context switches.

Furthermore, the Singularity design is flexible because each process is a closed entity and thus can have its own code, data structures, memory layout, runtime system, libraries, and garbage collector. The MMU is enabled, but only to map pages rather than to establish a separate protection domain for each process.

The Singularity operating system is written almost entirely in Sing#, a new type-safe language.

A key Singularity design principle is that it forbids dynamic process extensions. Among other consequences, the design does not permit loadable modules such as device drivers and browser plug-ins because they would introduce unverified foreign code that could corrupt the mother process. Instead, such extensions must run as separate processes, completely walled off and communicating by the standard IPC mechanism.

The microkernel

The Singularity operating system consists of a microkernel process and a set of user processes, all typically running in a common virtual address space. The microkernel controls access to hardware; allocates and deallocates memory; creates, destroys, and schedules threads; handles thread synchronization with mutexes; handles interprocess synchronization with channels; and supervises I/O. Each device driver runs as a separate process.

Although most of the microkernel is written in Sing#, a small portion is written in C#, C++, or assembler and must be trusted since it cannot be verified. The trusted code includes the hardware abstraction layer and the garbage collector. The hardware abstraction layer hides the low-level hardware from the system by hiding concepts such as I/O ports, interrupt request lines, direct memory access channels, and timers to present machine-independent abstractions to the rest of the operating system.

Interprocess communication

User processes obtain system services by sending strongly typed messages to the microkernel over point-to-point bidirectional channels. In fact, all process-to-process communication uses these channels. Unlike other message-passing systems, which have SEND and RECEIVE functions in some library, Sing# fully supports channels in the language, including formal typing and protocol specifications.

To make this point clear, consider this channel specification:

```
contract C1 {
  in message Request(int x) requires x > 0;
  out message Reply(int y);
  out message Error();

  state Start:
    Request? -> Pending;
  state Pending: one {
    Reply! -> Start;
    Error! -> Stopped;
  }
  state Stopped: ;
}
```

This contract declares that the channel accepts three messages, Request, Reply, and Error, the first with a positive integer as parameter, the second with any integer as parameter, and the third with no parameters. When used for a channel to a server, the Request messages go from the client to the server and the other two messages go the other way. A state machine specifies the protocol for the channel.

In the Start state, the client sends the Request message, putting the channel into the Pending state. The server can either respond with a Reply message or an Error message. The Reply message transitions the channel back to the Start state, where communication can continue. The Error message transitions the channel to the Stopped state, ending communication on the channel.

The heap

If all data, such as file blocks read from disk, had to go over channels, the system would be very slow, so an exception is made to the basic rule that each process's data is completely private and internal to itself. Singularity supports a shared object heap, but at each instant every object on the heap belongs to a single process. However, ownership of an object can be passed over a channel.

As an example of how the heap works, consider I/O. When a disk driver reads in a block, it puts the block on the heap. Later, the system passes the handle for the block to the user requesting the data, maintaining the single-owner principle but allowing data to move from disk to user with zero copies.

The file system

Singularity maintains a single hierarchical name space for all services. A root name server handles the top of the tree, but other name servers can be mounted on its nodes. In particular, the file system, which is just a process, is mounted on /fs, so a name like /fs/users/linda/foo could be a user's file. Files are implemented as B-trees, with the block numbers as the keys. When a user process asks for a file, the file system commands the disk driver to put the requested blocks on the heap. Ownership is then passed as described.

Singularity maintains a single hierarchical name space for all services.

Verification

Each system component has metadata describing its dependencies, exports, resources, and behavior. This metadata is used for verification. The system image consists of the microkernel, drivers, and applications needed to run the system, along with their metadata. External verifiers can perform many checks on the image before the system executes it, such as making sure that drivers do not have resource conflicts.

Verification is a three-step process:

- The compiler checks type safety, object ownership, channel protocols, and so on.
- The compiler generates Microsoft Intermediate Language, a portable JVM-like byte code that the verifier can check.
- MSIL is compiled to x86 code by a back-end compiler, which could insert runtime checks into the code (the current compiler does not do this though).

The point of redundant verification is to catch errors in the verifiers.

Each of the four different attempts to improve operating system reliability focuses on preventing buggy device drivers from crashing the system.

In the Nooks approach, each driver is individually hand wrapped in a software jacket to carefully control its interactions with the rest of the operating system, but it leaves all the drivers in the kernel. The paravirtual machine approach takes this one step further and moves the drivers to one or more machines distinct from the main one, taking away even more power from the drivers. Both of these approaches are intended to improve the reliability of existing (legacy) operating systems.

In contrast, two other approaches replace legacy operating systems with more reliable and secure ones. The multiserver approach runs each driver and operating system component in a separate user process and allows them to communicate using the microkernel's IPC mechanism. Finally, Singularity, the most radical approach, uses a type-safe language, a single address space, and formal contracts to carefully limit what each module can do.

Three of the four research projects—L4-based paravirtualization, Minix 3, and Singularity—use microkernels. It is not yet known which, if any, of these approaches will be widely adopted in the long run. Nevertheless, it is interesting to note that microkernels—long discarded as unacceptable because of their lower performance compared with monolithic kernels—might be making a comeback due to their potentially higher reliability, which many people now regard as more important than performance. The wheel of reincarnation has turned. ■

Acknowledgments

We thank Brian Bershad, Galen Hunt, and Michael Swift for their comments and suggestions. This work was supported in part by the Netherlands Organization for Scientific Research under grant 612-060-420.

References

1. V.R. Basili and B.T. Perricone, "Software Errors and Complexity: An Empirical Investigation," *Comm. ACM*, Jan. 1984, pp. 42-52.
2. T.J. Ostrand and E.J. Weyuker, "The Distribution of Faults in a Large Industrial Software System," *Proc. Int'l Symp. Software Testing and Analysis*, ACM Press, 2002, pp. 55-64.
3. A. Chou et al., "An Empirical Study of Operating System Errors," *Proc. 18th ACM Symp. Operating System Principles*, ACM Press, 2001, pp. 73-88.
4. M. Swift, B. Bershad, and H. Levy, "Improving the Reliability of Commodity Operating Systems," *ACM Trans. Computer Systems*, vol. 23, 2005, pp. 77-110.
5. M. Swift et al., "Recovering Device Drivers," *Proc. 6th Symp. Operating System Design and Implementation*, ACM Press, 2003, pp. 1-16.
6. R.P. Goldberg, "Architecture of Virtual Machines," *Proc. Workshop Virtual Computer Systems*, ACM Press, 1973, pp. 74-112.
7. J. LeVasseur et al., "Unmodified Device Driver Reuse and Improved System Dependability via Virtual Machines," *Proc. 6th Symp. Operating System Design and Implementation*, 2004, pp. 17-30.
8. J. Liedtke, "On Microkernel Construction," *Proc. 15th ACM Symp. Operating System Principles*, ACM Press, 1995, pp. 237-250.
9. H. Hartig et al., "The Performance of Microkernel-Based Systems," *Proc. 16th ACM Symp. Operating System Principles*, ACM Press, 1997, pp. 66-77.
10. J.N. Herder et al., "Modular System Programming in MINIX 3," Usenix; www.usenix.org/publications/login/2006-04/openpdfs/herder.pdf.

Andrew S. Tanenbaum *is a professor of computer science at Vrije Universiteit, Amsterdam. His research interests focus on operating systems and computer security. Tanenbaum received an SB from MIT and a PhD from the University of California, Berkeley. He is a Fellow of the IEEE and the ACM. Contact him at ast@cs.vu.nl.*

Jorrit N. Herder *is a PhD student in the Computer Systems Section of the Department of Computer Science at Vrije Universiteit, Amsterdam. His research focuses on the design and implementation of secure and reliable operating systems. Herder received an MSc in computer science from Vrije Universiteit. Contact him at jnherder@cs.vu.nl.*

Herbert Bos *is an assistant professor in the Computer Systems Section of the Department of Computer Science at Vrije Universiteit. His research interests include advanced networking technology, operating systems, and computer security. Bos received a PhD from the University of Cambridge. Contact him at bos@cs.vu.nl.*